

Plotting Directly with Matplotlib

What I'm Doing

In this lab I created and customized line, scatter, bar, histogram, pie, and subplot visualizations with Matplotlib using Canadian immigration data (1980–2013)

1. Setting Up

I imported NumPy, Pandas, and Matplotlib, then enabled the inline backend to render plots in the notebook.

```
import numpy as np
import pandas as pd

# Use the inline backend to generate plots within the notebook
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
```

2. Getting to Know the Dataset

The dataset covers international migration flows to Canada from 1980 to 2013, sourced from the United Nations. I fetch it directly from a hosted URL using `requests` and load it into a Pandas DataFrame.

```
import requests
import io

# Define the URL
URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/Canada.csv"

# Send a GET request to the URL
response = requests.get(URL)

# Convert the content into a BytesIO object
text = io.BytesIO(response.content)

# Read the CSV data into a pandas dataframe
df_can = pd.read_csv(text)
```

Preview data:

```
df_can.head()
```

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	...
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	...
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	...
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...	3626	...
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...	0	...
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...	0	...

5 rows × 39 columns

Checking data dimenions:

```
print(df_can.shape)
```

```
(195, 39)
```

195 rows (countries) and **39 columns** (country metadata + yearly immigration data from 1980 to 2013, plus a total column).

3. Cleaning the Data

I set the **Country** column as the index so I can easily look up countries using the `.loc` accessor.

```
df_can.set_index('Country', inplace=True)
df_can.head()
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2013	Total
Country													
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3436
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	1223
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	3626
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	0
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	0

5 rows × 38 columns

Notice now the country names now serve as indices. Let's confirm the shape and create a helper list of years for plotting.

```
print('data dimensions:', df_can.shape)

# Create a list of years from 1980 - 2013 as strings
years = list(map(str, range(1980, 2014)))

data dimensions: (195, 38)
```

4. Line Plot

A **line plot** shows the **relationship between two continuous variables over an interval**. I'll visualize the total immigration trend to Canada from 1980 to 2013.

First, I compute the yearly total immigrants by summing across all countries.

```
# Create df with only years columns from 1980 - 2013
df_line = df_can[years]

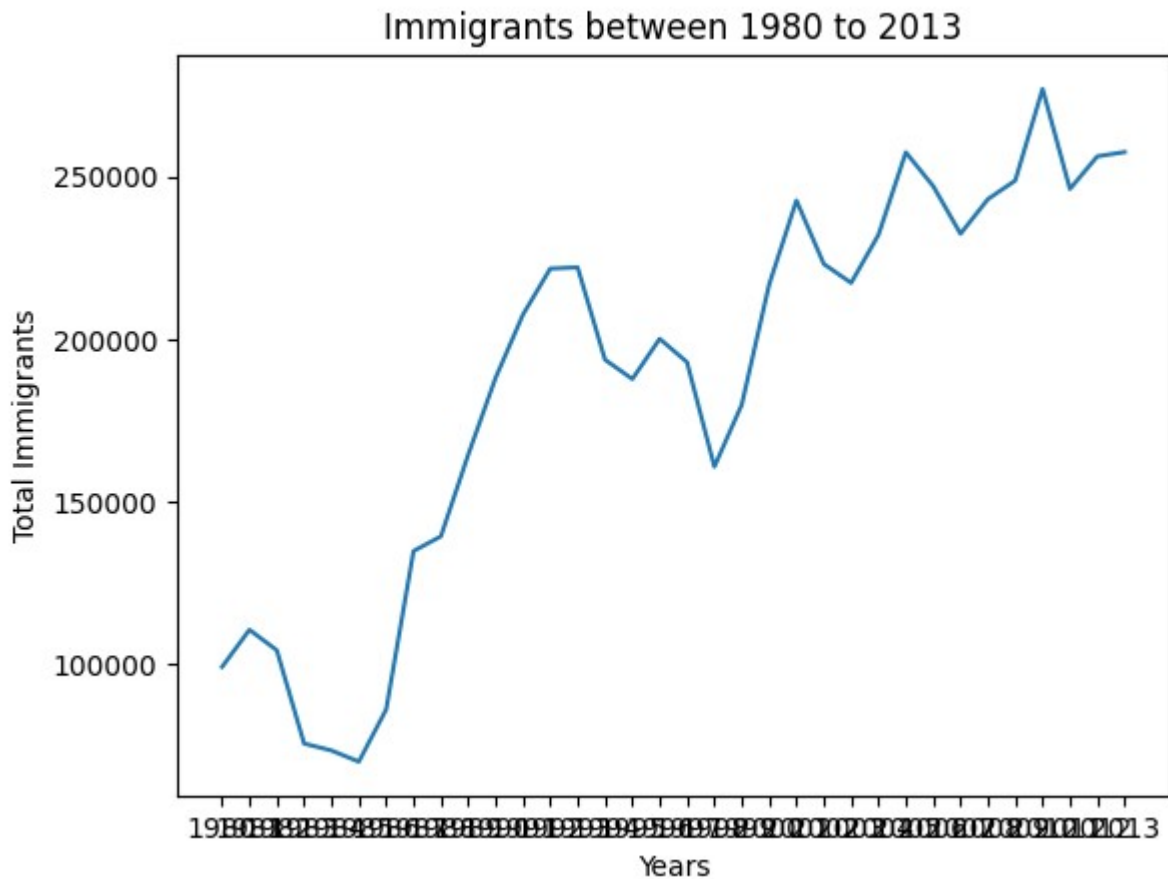
# Apply sum to get total immigrants year-wise
total_immigrants = df_line.sum()
total_immigrants
```

```
1980    99137
1981   110563
1982   104271
1983    75550
1984    73417
dtype: int64
```

Now let's plot this with Matplotlib.

```
fig, ax = plt.subplots()# Create figure and axes
ax.plot(total_immigrants)# Plot the Line

# Setting up the Title
ax.set_title('Immigrants between 1980 to 2013')
# Setting up the Labels
ax.set_xlabel('Years')
ax.set_ylabel('Total Immigrants')
plt.show()
```



Result: Line plot showing immigration trend from 1980 to 2013

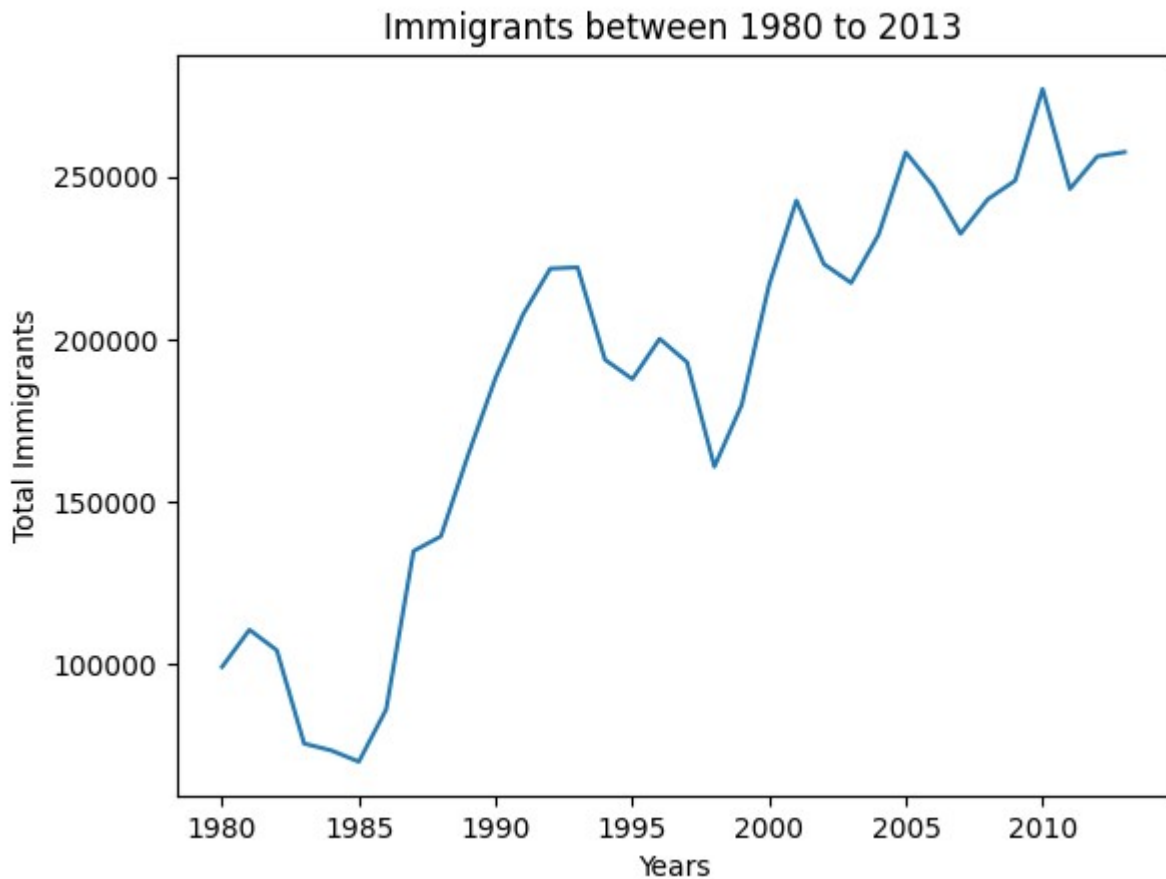
The x-axis shows the index (years), and the y-axis shows the total immigrant count. However, **the year labels didn't display cleanly** because the **index values are strings**. Let's **convert them to integers**.

```
# Create figure and axes
fig, ax = plt.subplots()

# Changing the index type to integer
total_immigrants.index = total_immigrants.index.map(int)

# Plot the line
ax.plot(total_immigrants)

ax.set_title('Immigrants between 1980 to 2013')
ax.set_xlabel('Years')
ax.set_ylabel('Total Immigrants')
plt.show()
```



Result: Line plot with integer-indexed years

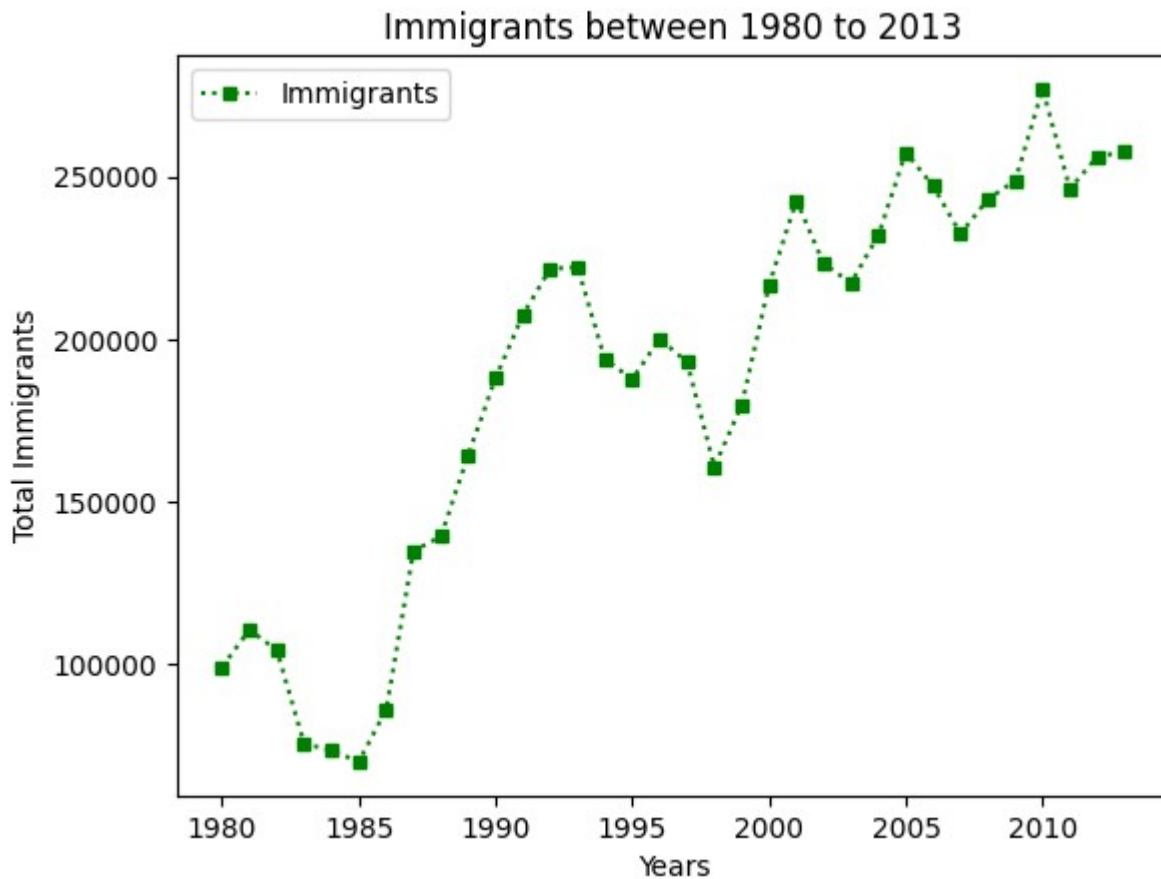
Much better. Now let's **customize the appearance** with markers, color, and line style.

```
fig, ax = plt.subplots() # Create figure and axes

# Changing the index type to integer
total_immigrants.index = total_immigrants.index.map(int)

# Customizing the appearance of Plot
ax.plot(total_immigrants,
        marker='s',           # Square markers
        markersize=5,         # Size of markers
        color='green',         # Line color
        linestyle="dotted")   # Dotted line style

# Setting up the Title
ax.set_title('Immigrants between 1980 to 2013')
ax.set_xlabel('Years')
ax.set_ylabel('Total Immigrants')
ax.legend(['Immigrants'])
plt.show()
```



Result: Customized line plot with green dotted line and square markers

Let's **add a grid**, a **legend**, and **adjust the axis limits**.

```
fig, ax = plt.subplots() # Create figure and axes
```

```
# Plot the Line
```

```
ax.plot(total_immigrants,  
        marker='s',  
        markersize=5,  
        color='green',  
        linestyle="dotted")
```

```
# Setting up the Title
```

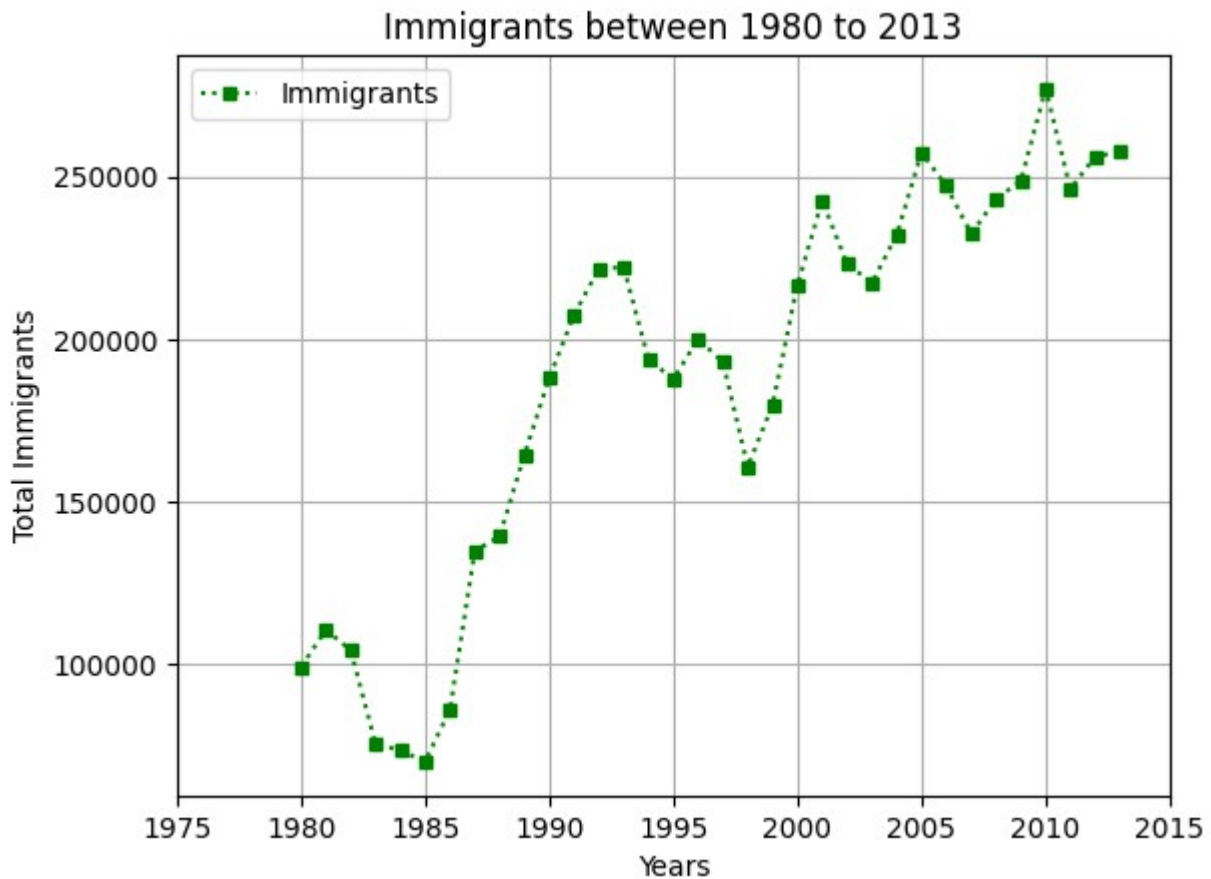
```
ax.set_title('Immigrants between 1980 to 2013')  
ax.set_xlabel('Years')  
ax.set_ylabel('Total Immigrants')
```

```
plt.xlim(1975, 2015) # Limits on x-axis
```

```
plt.grid(True) # Enabling Grid
```

```
plt.legend(["Immigrants"]) # Legend
```

```
plt.show()
```



Result: Final customized line plot with grid, legend, and adjusted x-limits

Exercise: Immigration from Haiti

In 2010, Haiti suffered a catastrophic magnitude 7.0 earthquake. Let's visualize how Canada's humanitarian response affected immigration from Haiti.

```
# Creating data for plotting
df_can.reset_index(inplace=True)
haiti = df_can[df_can['Country'] == 'Haiti']

# Creating haiti with only years columns from 1980 - 2013
# and transposing to get the result as a series
haiti = haiti[years].T

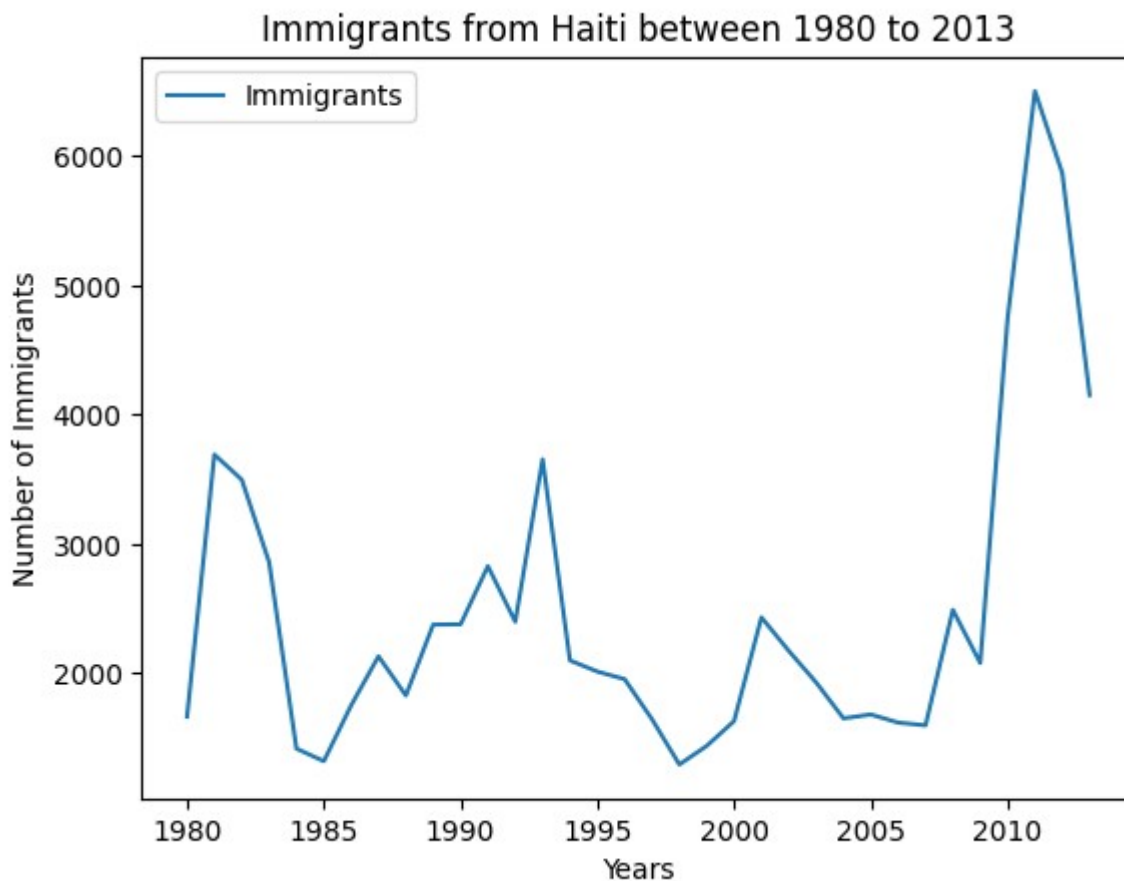
# Converting the index to type integer
haiti.index = haiti.index.map(int)

# Plotting the line plot on the data
fig, ax = plt.subplots()
ax.plot(haiti)

# Setting up the Title
ax.set_title('Immigrants from Haiti between 1980 to 2013')
ax.set_xlabel('Years')
ax.set_ylabel('Number of Immigrants')

# Legend
plt.legend(["Immigrants"])
```

```
plt.show()
```



Result: Line plot of immigration from Haiti from 1980 to 2013

The spike after 2010 is clear - Canada stepped up refugee acceptance. Let's annotate this point.

```
fig, ax = plt.subplots()
```

```
ax.plot(haiti)
```

```
ax.set_title('Immigrants from Haiti between 1980 to 2013')
```

```
ax.set_xlabel('Years')
```

```
# Alternative
```

```
#ax.set_xticks(list(range(n, m, s)))
```

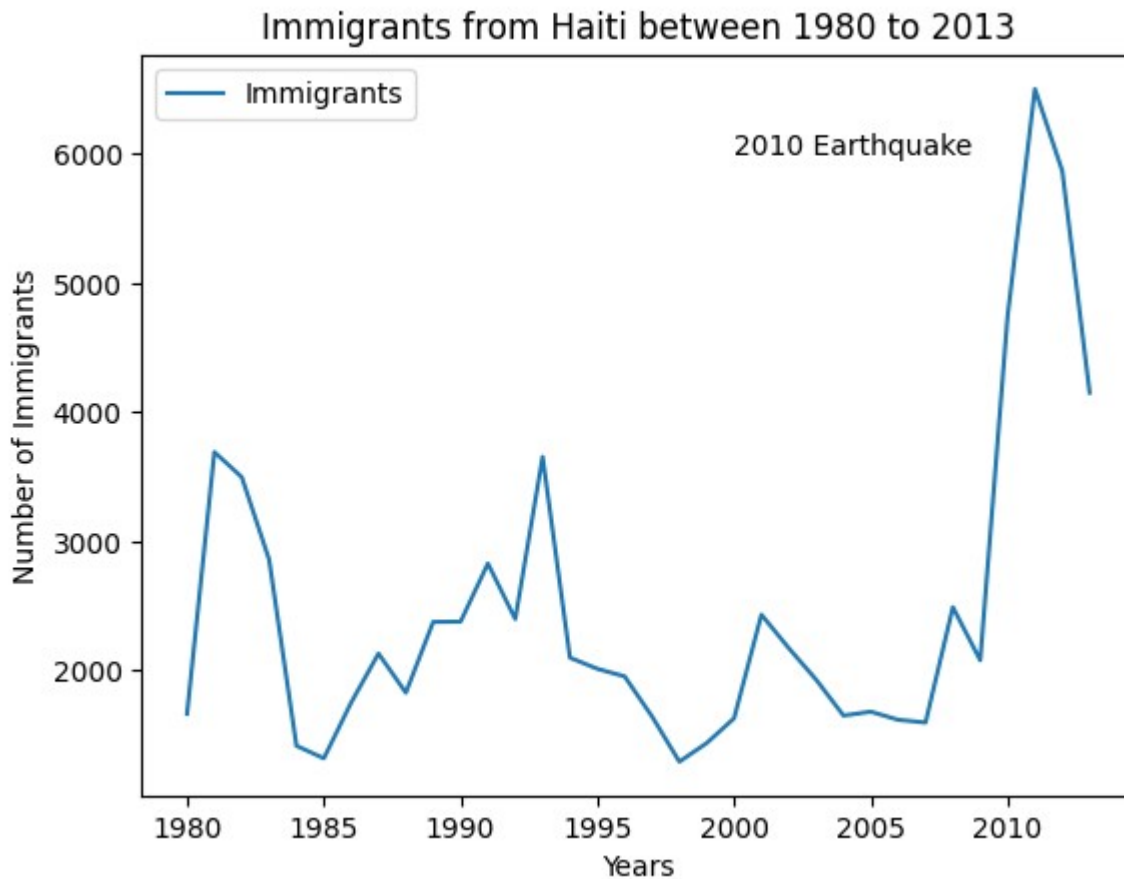
```
#ax.set_xticks(list(range(1980, 2015, 5)))
```

```
ax.set_ylabel('Number of Immigrants')
```

```
plt.legend(["Immigrants"])
```

```
ax.annotate('2010 Earthquake', xy=(2000, 6000))
```

```
plt.show()
```

Result: Line plot with annotation marking the 2010 Earthquake

5. Scatter Plot

A scatter plot **displays individual data points to reveal patterns between two continuous variables**. Let's create one for the same total immigration data.

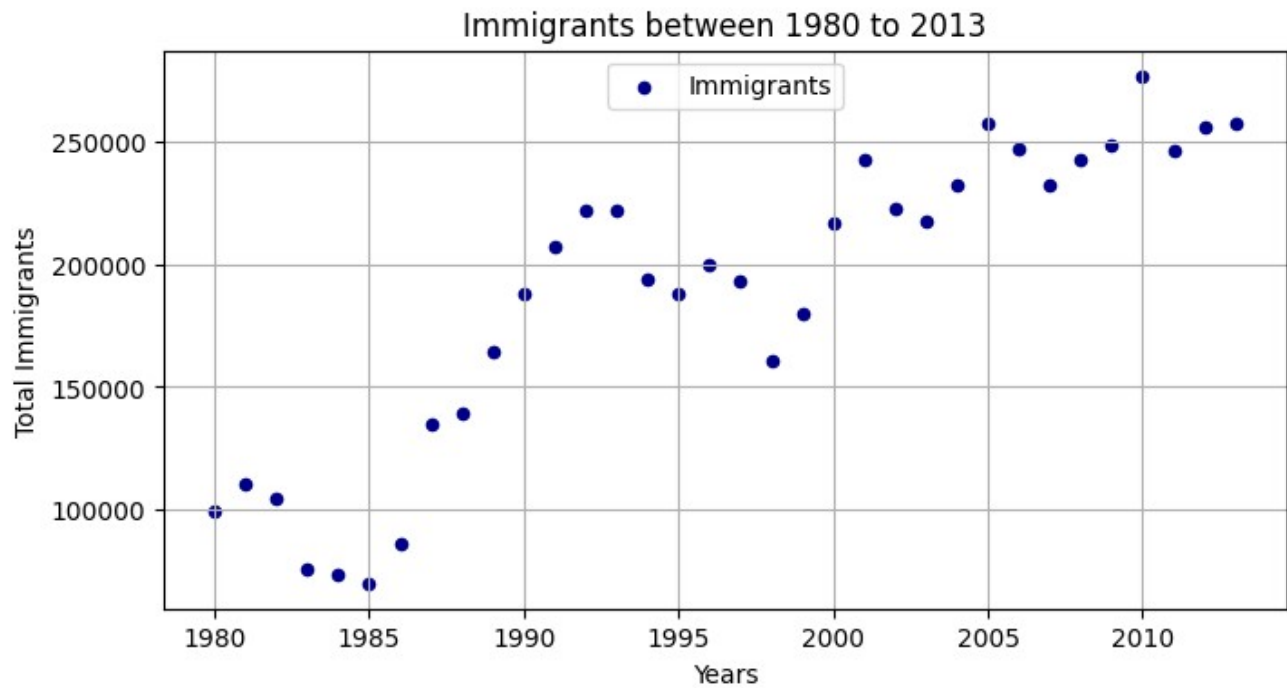
```
fig, ax = plt.subplots(figsize=(8, 4))

total_immigrants.index = total_immigrants.index.map(int)

# Customizing Scatter Plot
ax.scatter(total_immigrants.index, total_immigrants,
           marker='o',           # Circle markers
           s=20,                 # Marker size
           color='darkblue')     # Marker color

plt.title('Immigrants between 1980 to 2013')
plt.xlabel('Years')
plt.ylabel('Total Immigrants')
plt.grid(True)

# Legend at upper center of the figure
ax.legend(["Immigrants"], loc='upper center')
plt.show()
```

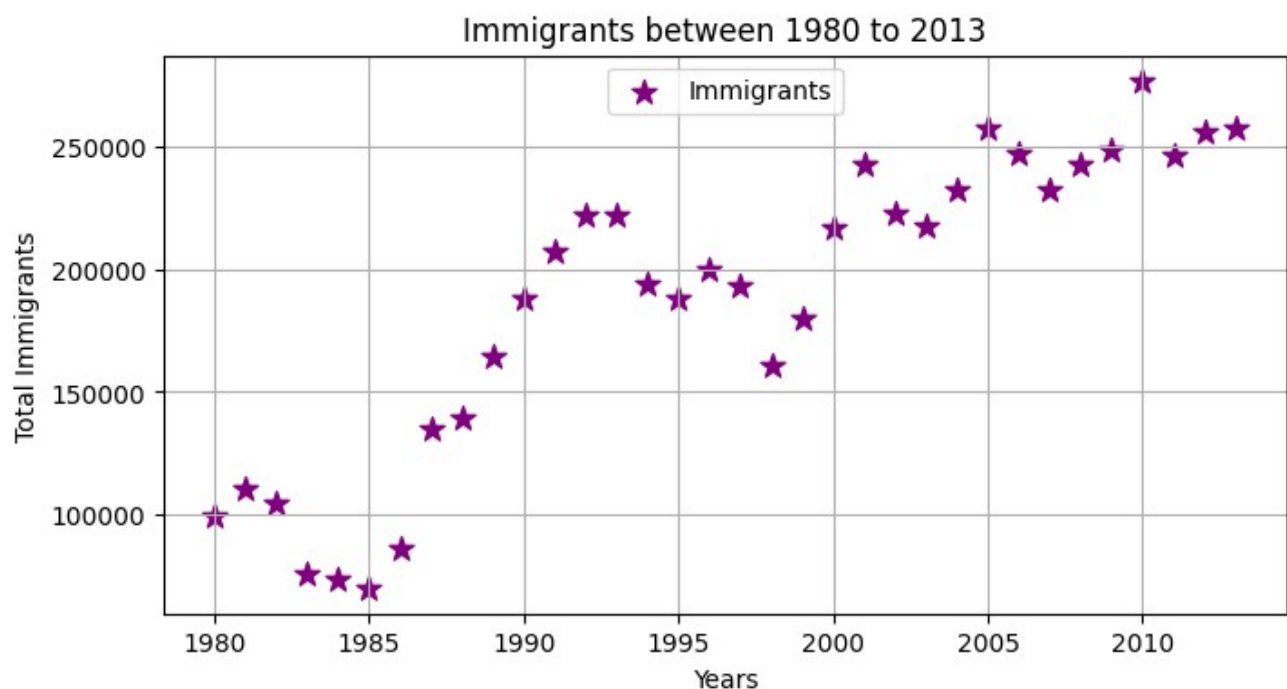


Result: Scatter plot showing immigration trend with dark blue dots

Each dot represents the total number of immigrants for a given year. The upward trend over time is clearly visible.

More markers variation

```
# Scatter plot with star markers
ax.scatter(total_immigrants.index, total_immigrants,
           marker='*',      # star shape
           s=100,          # bigger size for visibility
           color='purple')# playful color
```



Result: Using stars as markers bigger size.

6. Bar Plot

A bar plot uses **rectangular bars** to compare values across categories. Let's look at the **top 5 countries that contributed the most immigrants** to Canada from 1980 to 2013.

```
# Sorting the dataframe on 'Total' in descending order
df_can.sort_values(['Total'], ascending=False, axis=0, inplace=True)

# Get the top 5 entries with head function
df_top5 = df_can.head()

# Resetting the index
df_bar_5 = df_top5.reset_index()

# Creating a list of names of the top 5 countries
label = list(df_bar_5.Country)
label

['India',
 'China',
 'United Kingdom of Great Britain and Northern Ireland',
 'Philippines',
 'Pakistan']
```

The third country name is too long. Let's shorten it.

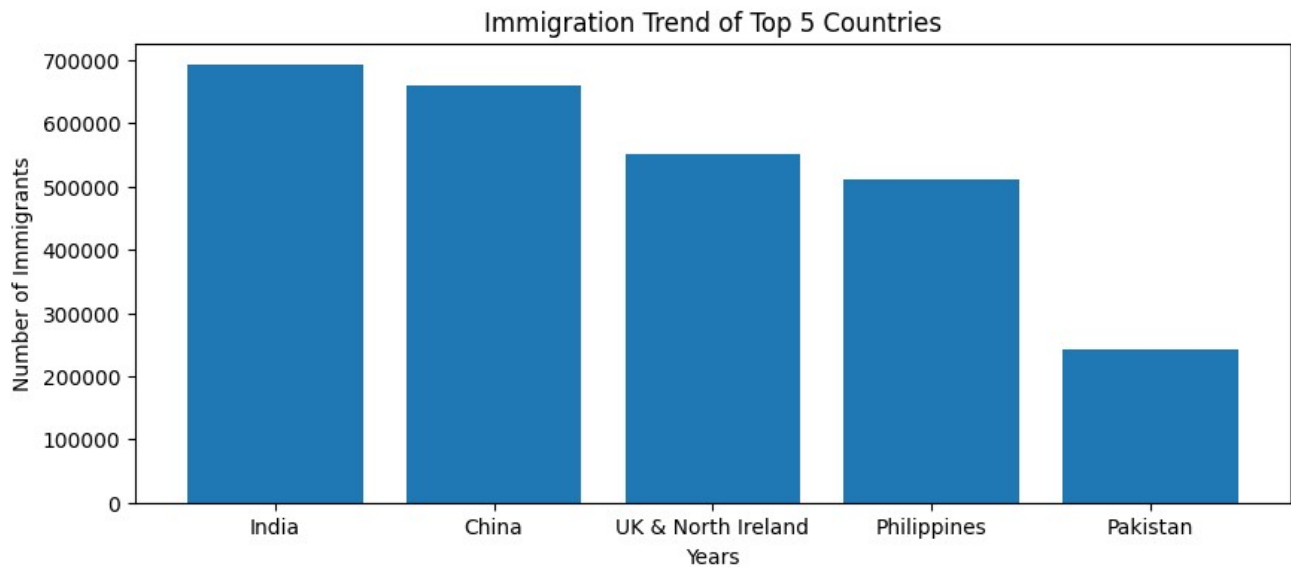
```
label[2] = 'UK & North Ireland'
label

['India', 'China', 'UK & North Ireland', 'Philippines', 'Pakistan']
```

Now let's plot the bar chart.

```
fig, ax = plt.subplots(figsize=(10, 4))

ax.bar(label, df_bar_5['Total'], label=label)
ax.set_title('Immigration Trend of Top 5 Countries')
ax.set_ylabel('Number of Immigrants')
ax.set_xlabel('Years')
plt.show()
```



Result: Bar plot showing top 5 countries by total immigration

Exercise: Bottom 5 Countries

Now let's create a bar plot for the **5 countries that contributed the least to immigration**.

```
# Sorting the dataframe on 'Total' in ascending order
df_can.sort_values(['Total'], ascending=True, axis=0, inplace=True)

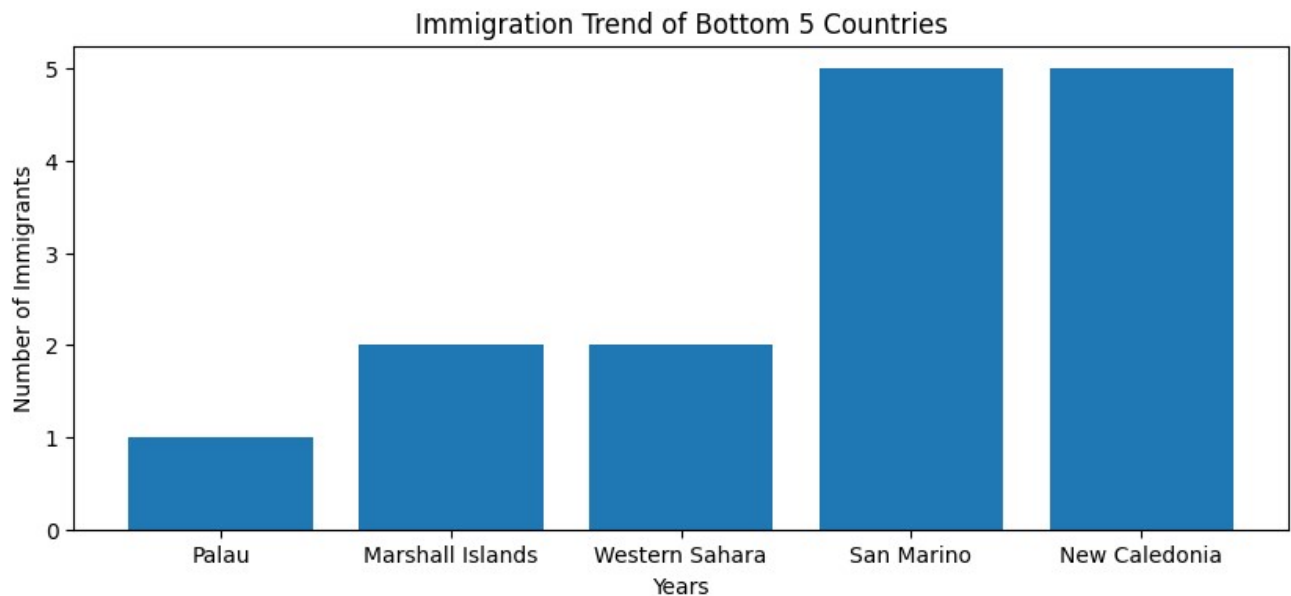
# Get the top 5 entries with head function
df_least5 = df_can.head()

# Resetting the index
df_least5_bar = df_least5.reset_index()

# Creating a list of names of the top 5 countries
label = list(df_least5_bar.Country)

fig, ax = plt.subplots(figsize=(10, 4))

ax.bar(label, df_least5_bar['Total'], label=label)
ax.set_title('Immigration Trend of Bottom 5 Countries')
ax.set_ylabel('Number of Immigrants')
ax.set_xlabel('Years')
plt.show()
```



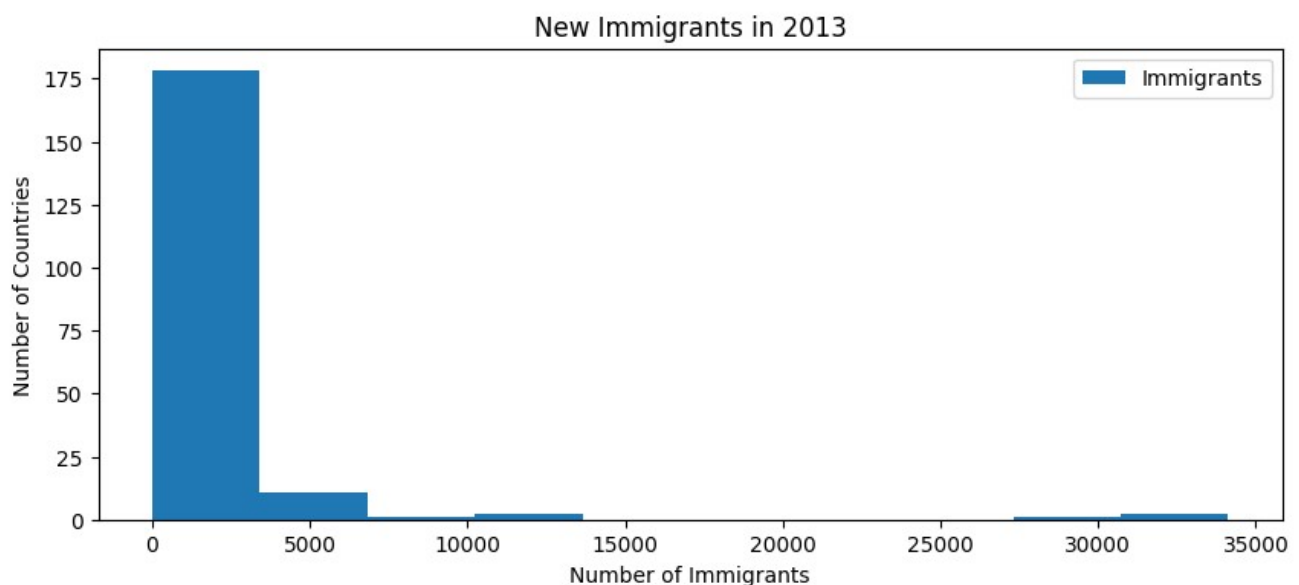
Result: Bar plot showing bottom 5 countries by total immigration

7. Histogram

A histogram shows the **frequency distribution of a numeric variable by grouping data into bins**. Let's see how many countries fell into each immigration range in **2013**.

```
df_country = df_can.groupby(['Country'])['2013'].sum().reset_index()
```

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(df_country['2013'])
ax.set_title('New Immigrants in 2013')
ax.set_xlabel('Number of Immigrants')
ax.set_ylabel('Number of Countries')
ax.legend(['Immigrants'])
plt.show()
```

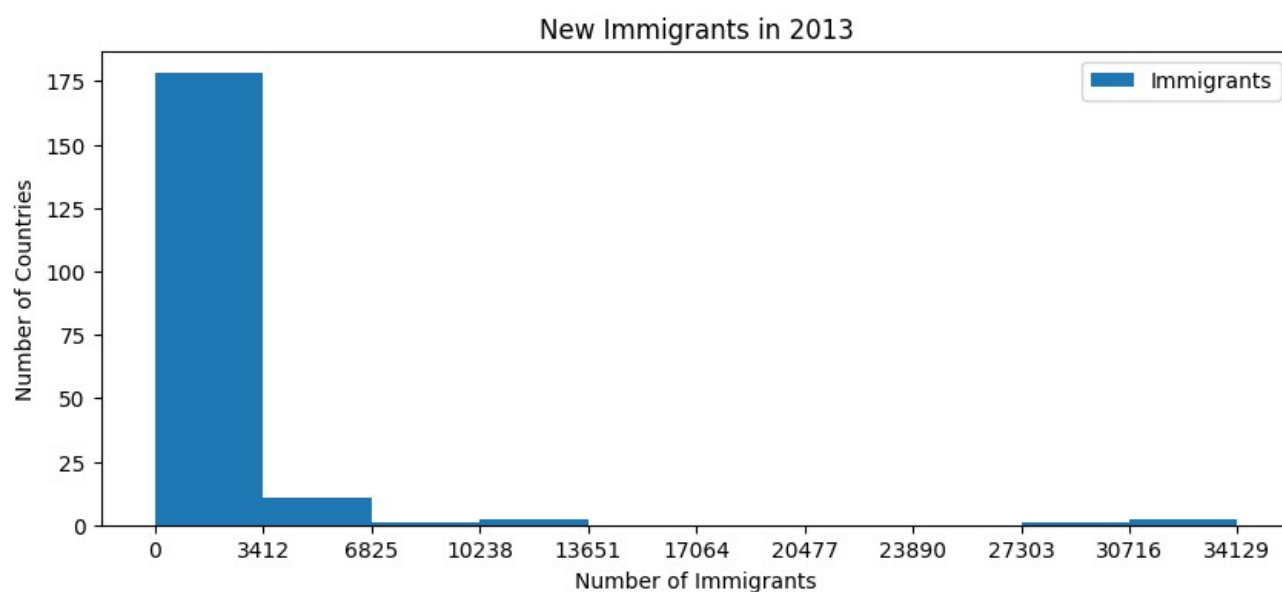


Result: Histogram of **new immigrants in 2013**

By **default**, Matplotlib splits the data into **10 bins**. Let's inspect the bin edges and counts, then use them as x-tick labels.

```
# Plot the bar
fig, ax = plt.subplots(figsize=(10, 4))
count = ax.hist(df_country['2013'])

ax.set_title('New Immigrants in 2013')
ax.set_xlabel('Number of Immigrants')
ax.set_ylabel('Number of Countries')
ax.set_xticks(list(map(int, count[1])))
ax.legend(['Immigrants'])
plt.show()
```



Result: Histogram with **bin edges as x-tick labels**

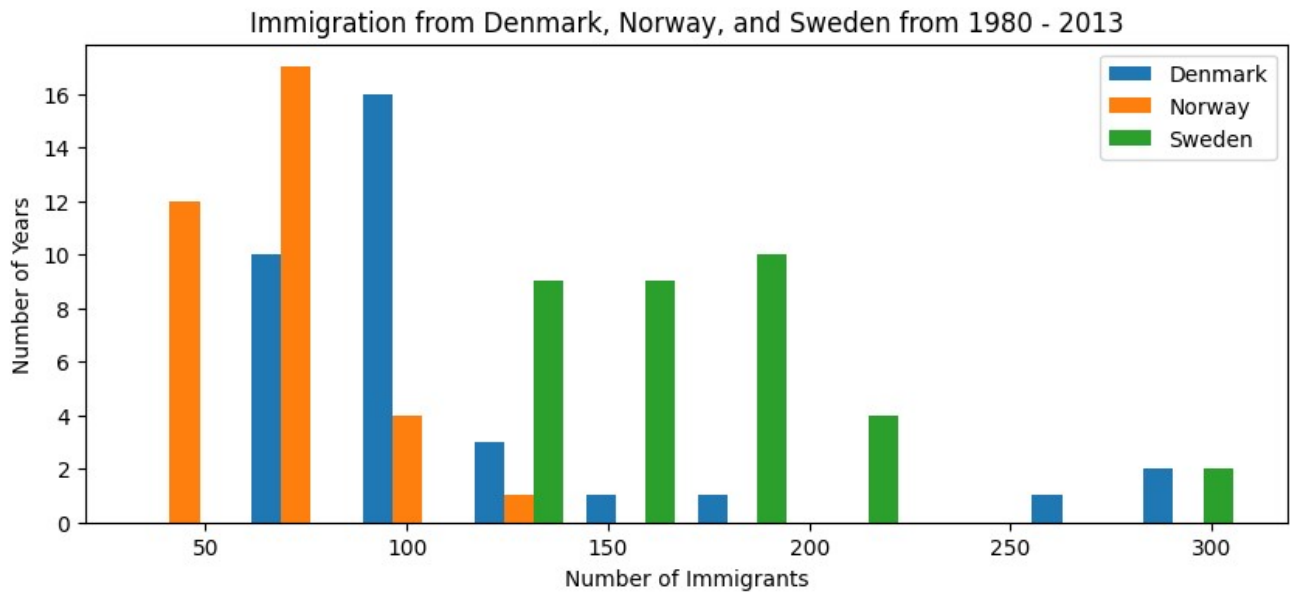
We can also **plot multiple histograms** together. Here I compare the immigration distributions for **Denmark, Norway, and Sweden** from 1980 to 2013.

```
df = df_can.groupby(['Country'])[years].sum()
df_dns = df.loc[['Denmark', 'Norway', 'Sweden'], years]
df_dns = df_dns.T
df_dns
```

Country	Denmark	Norway	Sweden
1980	272	116	281
1981	293	77	308
1982	299	106	222
1983	106	51	176
1984	93	31	128

Now plot the histogram

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(df_dns)
ax.set_title('Immigration from Denmark, Norway, and Sweden from 1980 - 2013')
ax.set_xlabel('Number of Immigrants')
ax.set_ylabel('Number of Years')
ax.legend(['Denmark', 'Norway', 'Sweden'])
plt.show()
```



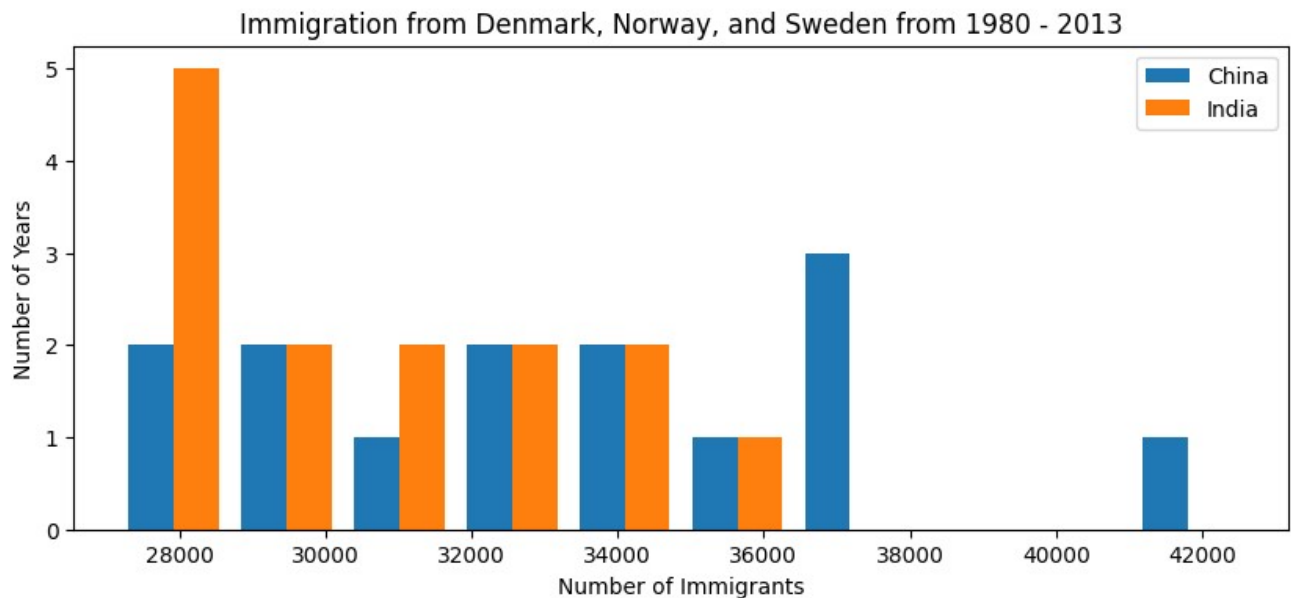
Result: Multiple histogram comparing Denmark, Norway, Sweden immigration

Exercise: China and India Histogram

Here I examine the immigration distribution for **China and India** for the years **2000 to 2013**.

```
df = df_can.groupby(['Country'])[years].sum()
y = list(map(str, range(2000, 2014)))
df_ci = df.loc[['China', 'India'], y]
df_ci = df_ci.T

fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(df_ci)
ax.set_title('Immigration from Denmark, Norway, and Sweden from 1980 - 2013')
ax.set_xlabel('Number of Immigrants')
ax.set_ylabel('Number of Years')
ax.legend(['China', 'India'])
plt.show()
```



Result: Histogram comparing immigration from China and India

8. Pie Chart

A pie chart represents **proportions of categories as sectors of a circle**. Let's see the distribution of **total immigrants for the years 1980 to 1985**.

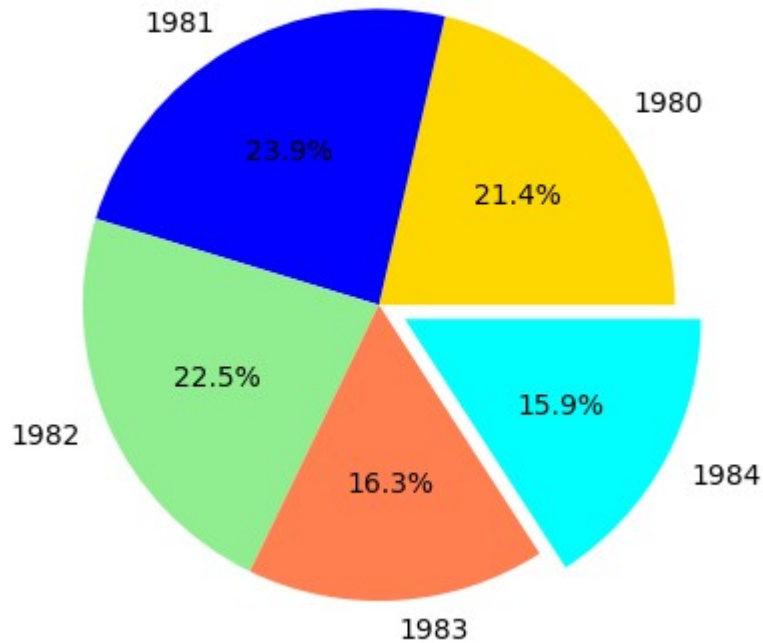
```
fig, ax = plt.subplots()

# Pie on immigrants
ax.pie(total_immigrants[0:5], labels=years[0:5],
       colors=['gold', 'blue', 'lightgreen', 'coral', 'cyan'],
       autopct='%1.1f%%',
       explode=[0, 0, 0, 0, 0.1]) # Explode to highlight the smallest slice

ax.set_aspect('equal') # Ensure pie is drawn as a circle

plt.title('Distribution of Immigrants from 1980 to 1985')
plt.show()
```


Distribution of Immigrants from 1980 to 1985



Result: Pie chart showing immigration distribution 1980-1985

Exercise: Continent-wise Pie Chart

Here I group the data by continent and visualize each continent's share of total immigration.

```
# Creating data for plotting pie
df_con = df_can.groupby('Continent')['Total'].sum().reset_index()
label = list(df_con.Continent)
label[3] = 'LAC'
label[4] = 'NA'
df_con
```

	Continent	Total
0	Africa	618948
1	Asia	3317794
2	Europe	1410947
3	Latin America and the Caribbean	765148
4	Northern America	241142
5	Oceania	55174

Next I plot the pie

```
fig, ax = plt.subplots(figsize=(10, 4))
```

```
# Pie on immigrants
ax.pie(df_con['Total'],
```

```

colors=['gold', 'blue', 'lightgreen', 'coral', 'cyan', 'red'],
autopct='%1.1f%%',
pctdistance=1.25)

```

```

ax.set_aspect('equal') # Ensure pie is drawn as a circle

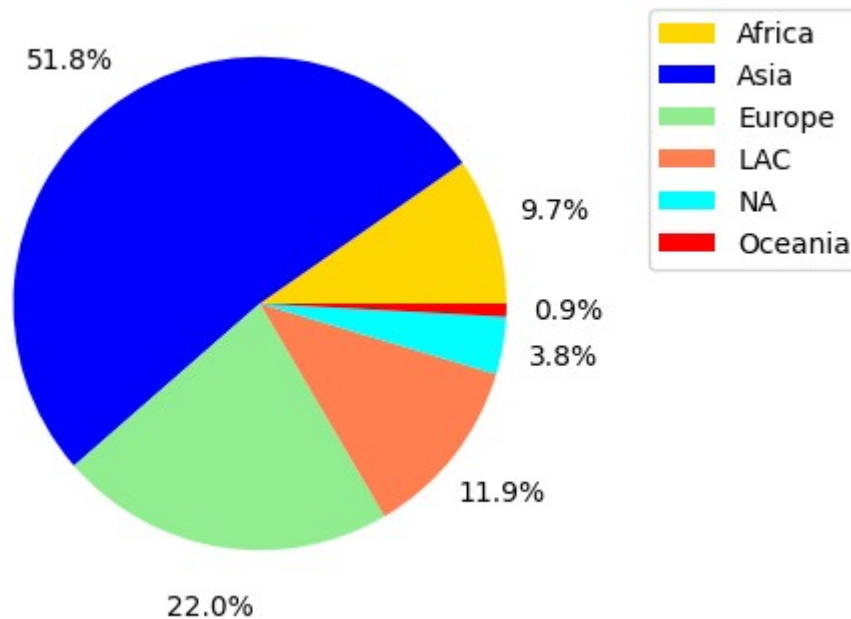
```

```

plt.title('Continent-wise distribution of immigrants')
ax.legend(label, bbox_to_anchor=(1, 0, 0.5, 1))
plt.show()

```

Continent-wise distribution of immigrants



Result: Pie chart showing continent-wise distribution of immigrants

9. Subplotting

Subplotting lets us **display multiple plots in a single figure**. Here I create a line plot and a scatter plot side by side.

Method 1: Using `plt.subplots()` with an array of axes

```

# Create a figure with two axes in a row
fig, axes = plt.subplots(1, 2, sharey=True)

# Plotting in first axes - the left one
axes[0].plot(total_immigrants)
axes[0].set_title("Line plot on immigrants")

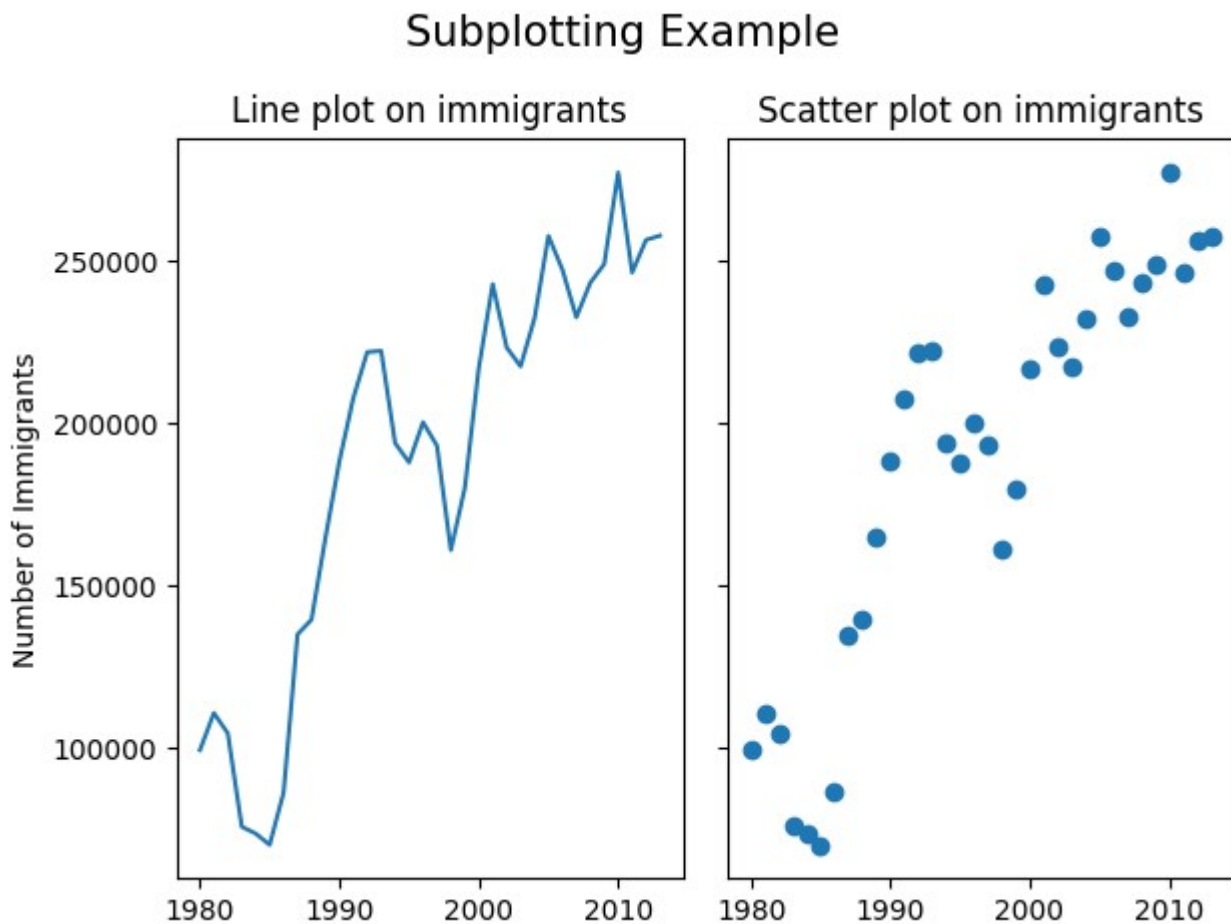
# Plotting in second axes - the right one
axes[1].scatter(total_immigrants.index, total_immigrants)
axes[1].set_title("Scatter plot on immigrants")

axes[0].set_ylabel("Number of Immigrants")

```

```
# Adding a Title for the Overall Figure
fig.suptitle('Subplotting Example', fontsize=15)

# Adjust spacing between subplots
fig.tight_layout()
plt.show()
```



Result: Subplot with line plot (left) and scatter plot (right) sharing y-axis

Method 2: Using `add_subplot()`

```
# Create a figure with Four axes - two rows, two columns
fig = plt.figure(figsize=(8, 4))

# Add the first subplot (top-left)
axs1 = fig.add_subplot(1, 2, 1)
axs1.plot(total_immigrants)
axs1.set_title("Line plot on immigrants")

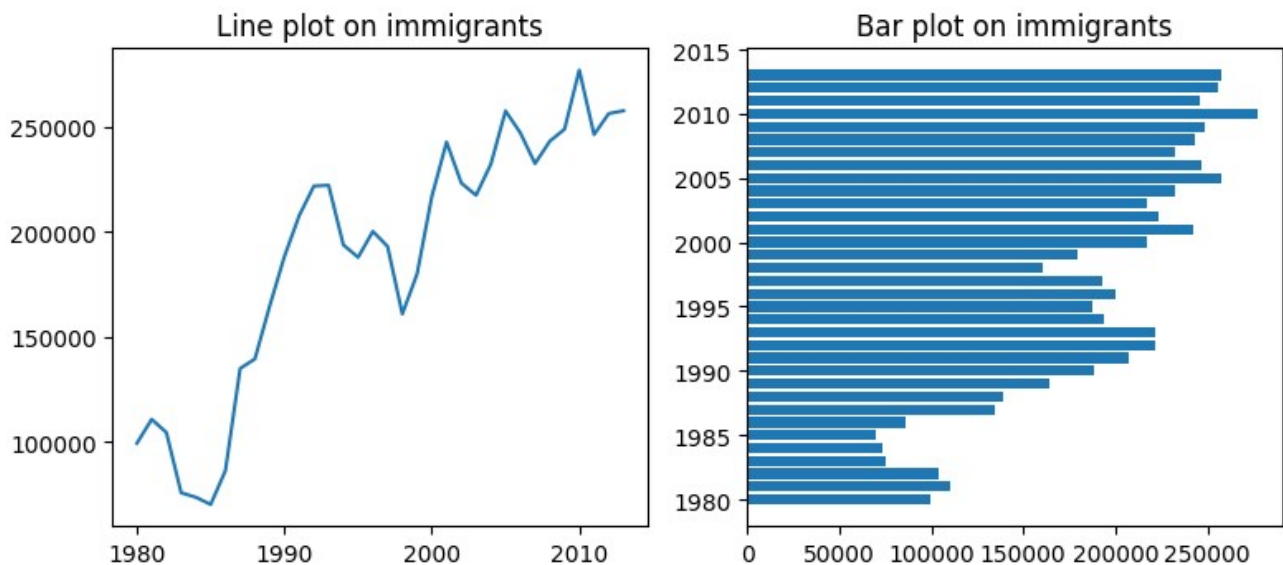
# Add the second subplot (top-right)
axs2 = fig.add_subplot(1, 2, 2)
axs2.barh(total_immigrants.index, total_immigrants) # Horizontal bar plot
axs2.set_title("Bar plot on immigrants")

# Adding a Title for the Overall Figure
fig.suptitle('Subplotting Example', fontsize=15)

# Adjust spacing between subplots
```

```
fig.tight_layout()
plt.show()
```

Subplotting Example



Result: Subplot with line plot (left) and horizontal bar plot (right)

Exercise: 2×2 Subplot Grid

Now I combine four different plot types from this lab into a single 2×2 figure.

```
# Create a figure with Four axes - two rows, two columns
fig = plt.figure(figsize=(10, 10))

# Add the first subplot (top-left)
ax1 = fig.add_subplot(2, 2, 1)
ax1.plot(total_immigrants)
ax1.set_title('Plot 1 - Line Plot')

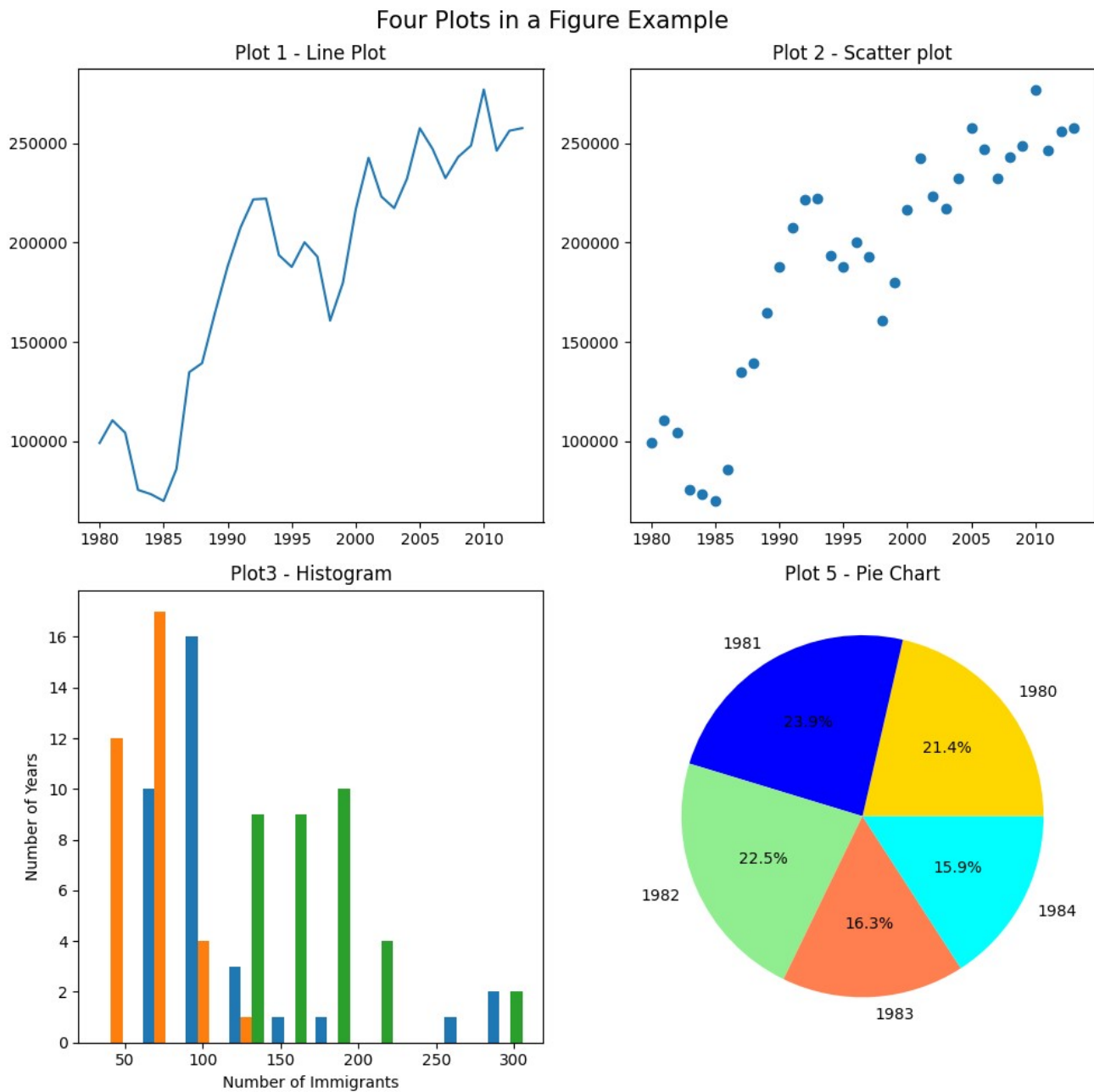
# Add the second subplot (top-right)
ax2 = fig.add_subplot(2, 2, 2)
ax2.scatter(total_immigrants.index, total_immigrants)
ax2.set_title('Plot 2 - Scatter plot')

# Add the third subplot (bottom-left)
ax3 = fig.add_subplot(2, 2, 3)
ax3.hist(df_dns)
ax3.set_title('Plot 3 - Histogram')
ax3.set_xlabel('Number of Immigrants')
ax3.set_ylabel('Number of Years')

# Add the fourth subplot (bottom-right)
ax4 = fig.add_subplot(2, 2, 4)
ax4.pie(total_immigrants[0:5], labels=years[0:5],
        colors=['gold', 'blue', 'lightgreen', 'coral', 'cyan'],
        autopct='%1.1f%%')
ax4.set_aspect('equal')
ax4.set_title('Plot 4 - Pie Chart')
```

```
# Adding a Title for the Overall Figure
fig.suptitle('Four Plots in a Figure Example', fontsize=15)

# Adjust spacing between subplots
fig.tight_layout()
plt.show()
```



Result: 2x2 subplot grid with line, scatter, histogram, and pie charts

That's the full walkthrough. I imported and cleaned the Canadian immigration dataset, then used Matplotlib to build and customize line, scatter, bar, histogram, pie, and subplot visualizations.